Проверка сокращённых микропрограмм ядра

Запуск из корня репозитория:

```
tests/run_core_native_tests.sh
MK61_TEST_SANITIZERS=1 tests/run_core_native_tests.sh
```

Тест включён в `tests/run_all_tests.sh`. Он собирает настоящее ядро с таблицами во Flash и в вытесняемом SRAM, в каждом случае проверяя обычное и расширенное кольцо. Сокращения сравниваются с исходным скалярным декодером; упакованный селектор AMK в этом тесте отключён для независимости сравнения. Его совместную работу с сокращениями проверяет существующий `tests/run_mk_math_tests.sh`.

Что проверяется

Для каждого из шести сокращений ИК1306 выполняется полный перебор `R36`, `R39`, `S` от 0 до F и входных битов `L`, `P`. Четыре окружающих состояния варьируют остальные регистры, `S1`, `T`, `MOD`, `flag_FC` и содержимое кольца. Это 393 216 случаев на режим кольца, 786 432 на расположение таблиц. Такой перебор исчерпывает перечисленные входы, но не всё пространство состояний калькулятора.

Сокращение	Работа декодера для сравнения
ИК1306, тело 00, участок 1	27 микротактов
ИК1306, тело 00, участок 2	9 микротактов
ИК1306, тело 06, участок 3	6 микротактов, прибавление S
ИК1306, тело 07, участок 3	6 микротактов, инкремент
ИК1306, тело 09, участок 3	6 микротактов, установка адреса 01
ИК1306, тело 40, участок 1	27 микротактов, чтение сигнала ожидания

После исполнения сравниваются все 672 байта снимка: состояния трёх чипов, кольцо, указатели, регистры, флаги и состояние обработки команд. Допустимо только точное совпадение, без числового допуска и без сравнения с `libm`.

В сценариях работы оба пути запускаются из одного снимка. Декодер записывает последовательность состояний после выборки ПЗУ, после микротактов 0–26, 27–35, 36–41 и после продвижения кольца. Сокращённый путь должен воспроизвести её точно. Сравниваются также события зарегистрированных обработчиков ПЗУ и команд, порядок BEFORE/AFTER, источник и номер команды, программные остановки, флаг уступки управления и состояние вызова обработчиков. Следующий шаг всегда продолжается из состояния исходного декодера.

Сценарии включают арифметику, стек, запись/чтение регистра, `atan(10)`, `sin`, `exp` во всех трёх единицах угла, деление на ноль и выход через Cx, остановку/повторную остановку/продолжение программы, CALL/RETURN/JMP, удержание и отпускание клавиш и воспроизводимую последовательность коротких нажатий с префиксами F/K. Проверяется, что каждый сокращённый путь действительно вызывался, оба источника команд дали BEFORE/AFTER, а обработчики всех трёх чипов были задействованы.

Ожидание ИК1306

Первая подтверждённая точка находится по адресу ПЗУ ИК1306 `01`, слово `00060040`. Её первый участок, тело `40`, читает восемь разрядов кольца на микротактах 1, 4, 7, 10, 13, 16, 19 и 22. Затем последний прочитанный разряд преобразуется в бит пробуждения: F даёт `S = L = 1`, остальные значения — 0. Уже существующее сокращение тела `06` третьего участка прибавляет S к адресу ПЗУ: чип остаётся на `01` или переходит на `02`.

Новая реализация упрощает декодирование тела `40`, сохраняя чтения кольца, запись R при `MOD = 0`, флаги и порядок исполнения всех трёх чипов на исходных микротактах. Продвижение кольца остаётся прежним.

Отдельная проверка перебирает начальный и новый разряды 0–F, оба состояния MOD, восемь точек чтения и изменение входа за такт до, на и через такт после чтения: 12 288 переходов на режим кольца. Все 672 байта сравниваются после каждого микротакта, затем проверяется следующий адрес ПЗУ после полного ожидания.

Проверка самих проверок

В тестовое исполнение отдельно вносятся девять дефектов: пропущенный сброс P, потерянный перенос, испорченный L при установке адреса, выбор сокращения для чужого тела, временное повреждение P на границе участка, пропущенный сдвиг кольца, пропущенная первая запрошенная программная остановка, запоздалое чтение сигнала ожидания и задержка пробуждения. Каждый дефект обязан вызвать несовпадение.

Для временного повреждения P дополнительно проверяется, что к концу большого шага состояние снова совпало. Это пример ошибки, которую прежняя проверка только после `step()` пропустила бы. Исполнение с повреждённым транспортом прерывается после обнаружения, до возможного выхода указателей за кольцо.

Что это позволяет утверждать

Тесты проверяют сохранение поведения существующего декодера. Эталоном калькулятора остаётся физический МК-61. Фикстура `atan(10) = 1.4711276` здесь фиксирует уже наблюдавшийся результат нашего декодера в радианах; она не обозначает новый замер на приборе. Общие ошибки исходного и ускоренного пути таким сравнением не обнаруживаются.

Проверка семантики событий выявила и теперь строго проверяет исправление существовавшей ошибки: для программы `53 06 51 09 00 00 01 52 00 50` обработчик границ раньше сообщал адреса `0, 6, 7, 2, 3, 9`. Адрес 3 содержит операнд JMP и ошибочно считался командой. После исправления обязательная последовательность равна `0, 6, 7, 2, 9`; события BEFORE/AFTER не вызываются на операнде, а AFTER перехода происходит при достижении следующей команды.

Дополнительно проверяются девять прямых и условных переходов при X = −1, 0, 1, переход на собственный операнд, JMP внутри пяти вложенных CALL и восстановление снимка с ожидающим операндом. Состояние ожидающего операнда занимает один байт рабочей RAM и упаковано в свободные биты снимка; его размер остался 672 байта.

Адреса всех девяти точек из исходного предложения не были заданы; найденная точка ИК1306:01 исследована отдельно. Проверки не подтверждают безопасность пропуска других ожиданий. Для каждой следующей точки нужны её адрес, читаемые сигналы, побочные эффекты и граница возобновления, затем такие же проверки смены сигналов и транспорта.

Тестовый файл включает реализацию ядра напрямую, чтобы вызывать настоящий декодер и сокращения без расширения публичного API. Наблюдение включается только макросом `MK61_CORE_TEST_BOUNDARY` и дополнительно запрещено при `ARDUINO`; тестовые снимки и трассы не входят в прошивку. При добавлении только точек наблюдения ARM-объекты ядра до и после изменения были побайтово одинаковы. В первоначальном варианте на STM32F411, GCC 14.2.1, `-Os` исправление границ добавило 88 байт кода и 1 байт BSS. Сокращение ожидания добавило ещё 256 байт кода без расхода RAM. Разница в 256 байт подтверждена полными сборками одной конфигурации Classic V3 UC1609: 330 264 байта с пятью сокращениями, 330 520 с шестью; глобальные переменные в обеих занимают 37 144 байта.

Замер на STM32F411

5 сентября 2026 года на подключённом Classic V3 UC1609 выполнен A/B-замер двух прошивок с одинаковыми настройками сборки. Обе содержат исправление границ переходов; единственное различие в исполняемом коде — включение сокращения тела `40`. В каждой прошивке команда `prof core 80` выполнена пять раз. Она каждый раз начинает из одинакового начального состояния ядра, измеряет 80 вызовов `core.step` счётчиком DWT и проверяет точное восстановление сохранённого контекста (`restored=1`).

Сборка	Размер Flash, байт	Средние такты на шаг в пяти повторах	Медиана
Пять сокращений, `2BD95C98`	330 264	3 555 330; 3 555 335; 3 555 305; 3 555 346; 3 555 307	3 555 330
Шесть сокращений, `3224E08A`	330 520	3 102 394; 3 102 334; 3 102 390; 3 102 407; 3 102 294	3 102 390

Затраты на шаг в этом сценарии уменьшились на 12,74%, ускорение — 1,146 раза за 256 байт Flash. Это замер ядра из начального состояния без запущенной программы; прирост скорости произвольных вычислений отдельно не измерялся.

До перепрошивки сохранены все 512 КиБ внутренней Flash и проверена CRC исходной прошивки `4AC27AA8`. После измерений полный образ восстановлен; повторное чтение совпало с резервной копией побайтово, а команда `identity` подтвердила запуск исходной сборки. Прошивка перезапускала прибор, поэтому содержимое его прежней рабочей RAM не сохранялось.

Запас Flash в конфигурации F401

Проверка `f401-capability-ws-usb-graphics` выявила нехватку обязательного запаса Flash: первоначальная реализация занимала 261 712 из 262 144 байт, оставляя 432 байта при минимуме 512. Та же конфигурация и размер воспроизведены локально.

Обработка CALL/JMP теперь использует общий расчёт адреса операнда; длина команды определяется сравнениями вместо перебора таблицы. Удалены дублирующие проверки состояния. Полный перебор 256 кодов проверяет сохранение их длины, а тесты границ и снимков продолжают проверять обработку переходов.

Изменение сэкономило 80 байт в этой сборке F401: образ занимает 261 632 байта, запас равен 512 байтам. Порог проверки остался прежним. При прежней компиляции ARM-объекта F411 `-Os` размер кода уменьшился с 24 801 до 24 725 байт; итоговый прирост к исходному ядру — 268 байт кода и 1 байт BSS. Снимок занимает прежние 672 байта. Приведённый выше аппаратный замер относится к сборкам до этого упрощения обработки команд.